Homework 4 Submission

Use this template to record your answers for Homework 4. Add your answers using LaTeXand then save your document as a PDF to upload to Gradescope. You are required to use this template to submit your answers. **You should not alter this template in any way** other than to insert your solutions. You must submit all 9 pages of this template to Gradescope. Do not remove the instructions page(s). Altering this template or including your solutions outside of the provided boxes can result in your assignment being graded incorrectly.

You should also export your code as a .py file and upload it to the **separate** Gradescope coding assignment. Remember to mark all teammates on **both** assignment uploads through Gradescope.

Instructions for Specific Problem Types

On this homework, you must fill in blanks for each problem. Please make sure your final answer is fully included in the given space. **Do not change the size of the box provided.** For short answer questions you should **not** include your work in your solution. Only provide an explanation or proof if specifically asked.

Fill in the blank: What is the course number?

10-703

Problem 0: Collaborators

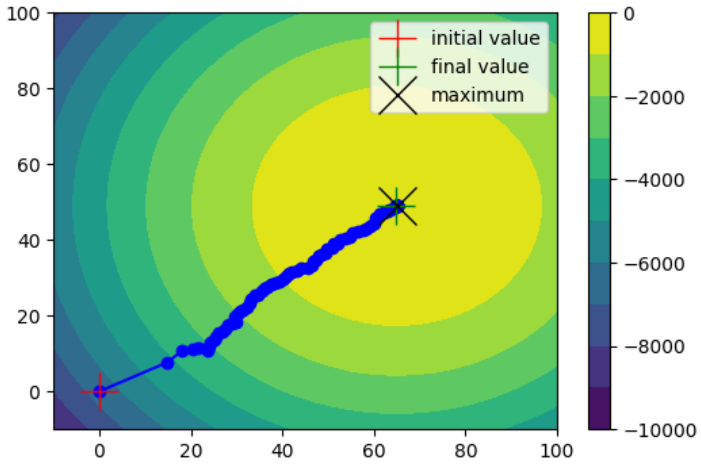
Enter your team members' names and Andrew IDs in the boxes below. If you worked in a team with fewer than three people, leave the extra boxes blank.

Name 1:	Arnav Goyal	Andrew ID 1:	arnavgoy
Name 2:	Vineeth Bhat	Andrew ID 2:	vineethb
Name 3:	Kushal Kejriwal	Andrew ID 3:	kkejriwa

Problem 1: CMA-ES (24 pts)

1.1 Plot of CMA-ES on simple objective function (10 pts)

Solution



1.2 RL reward of fixed policies (4 pts)

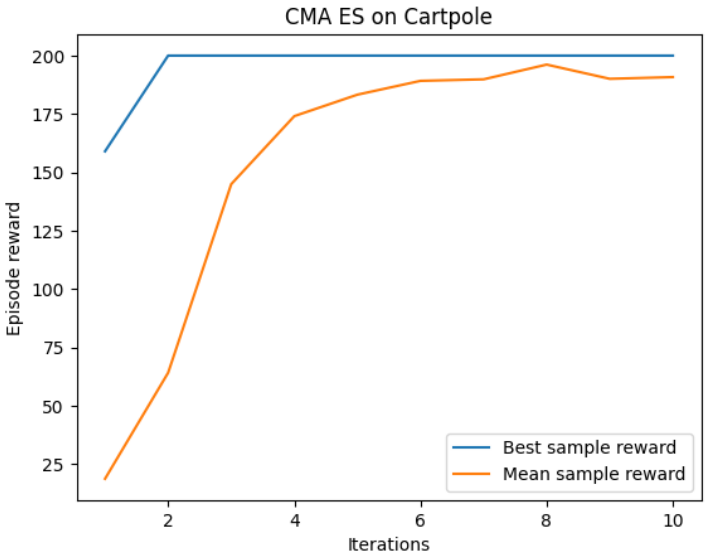
$x = (-1, -1, -1, -1, -1)$: 15.6

$x = (1, 0, 1, 0, 1)$: 14.59

$x = (0, 1, 2, 3, 4)$: 9.425

1.3 Plot of CMA-ES on Cartpole (10 pts)

Solution

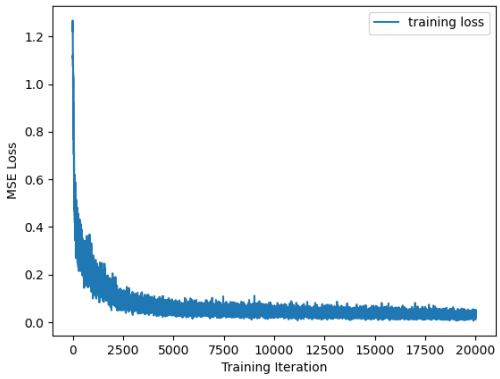


Problem 2: Imitation Learning (62 pts)

Problem 2.1: BC (14 pts)

2.1.1 Loss Plot + Final Loss Value BC (6 pts)

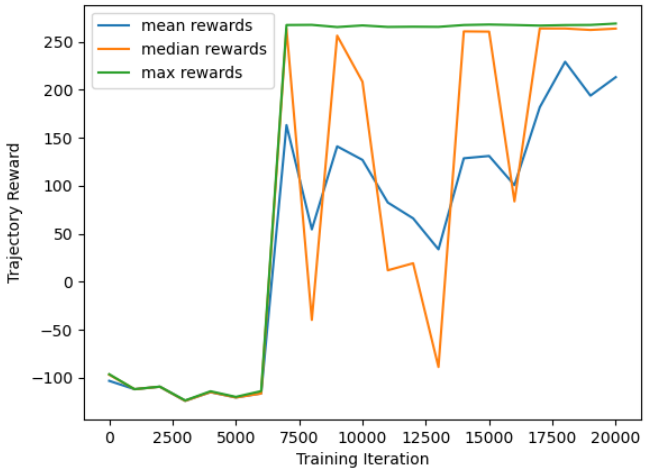
Solution



Final loss: 0.0123

2.1.2 Rewards Plot BC (6 pts)

Solution



2.1.3 GIF link BC (2 pts)

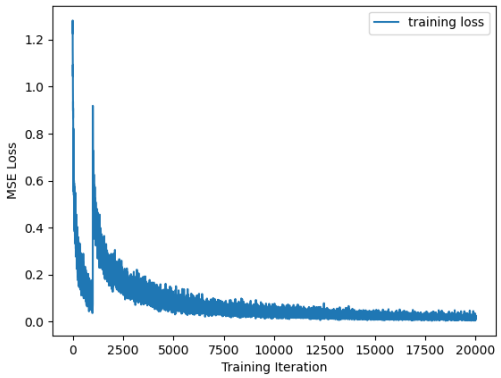
Solution

https://drive.google.com/file/d/1Z3-S1BLQdJrHZxqDcdyWtXMBsSGnJxWQ/view?usp=sharing

Problem 2.2: DAgger (18 pts)

2.2.1 Loss Plot DAgger (6 pts)

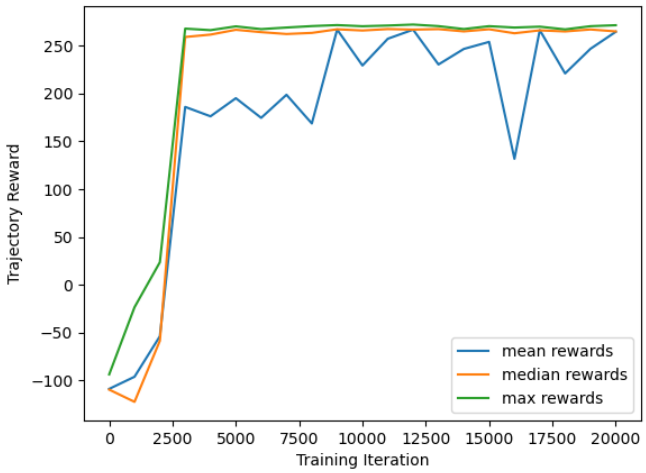
Solution



Final loss: 0.0168

2.2.2 Rewards Plot DAgger (6 pts)

Solution



2.2.3 GIF link DAgger (2 pts)

Solution

https://drive.google.com/file/d/1Tz_Nix4edSxUjCrV3Y8WdhgI3HLD5d7Q/view?usp=sharing

2.2.4 Compare DAgger training with BC (written, 4 pts)

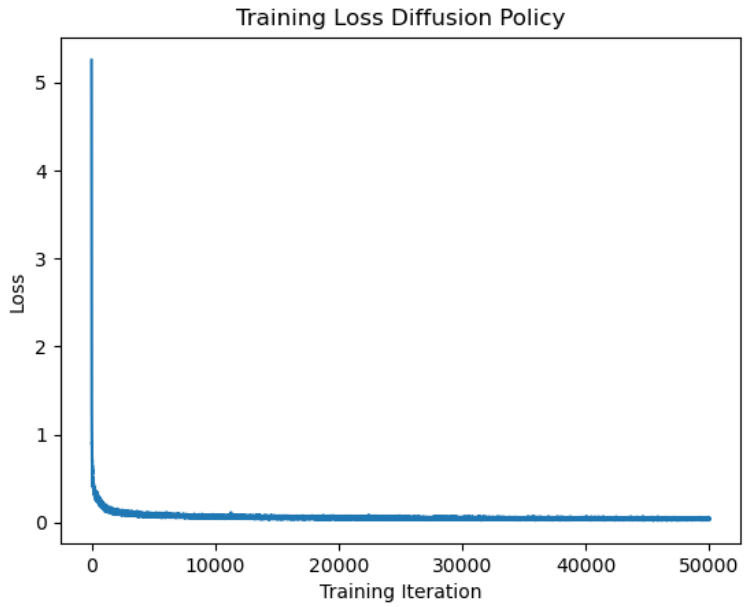
Solution

DAgger worked better. We did 5 independent runs over both DAgger and plain BC, and we observed that we learned how to walk in all 5 DAgger runs (final mean reward 240–265; close to expert's 261–266) while only in 3 of the BC runs (final mean reward 193–213), despite similar training losses. BC only sees states from expert trajectories, so a small action error takes the walker to states the expert never visited, where its errors compound until it falls. With per-step error ϵ over horizon T, BC's expected error grows as $O(\epsilon T^2)$. DAgger rolls out the learner, labels the states it actually visits with the expert's actions, and aggregates them, so the policy learns how to recover from its own mistakes and its error grows only as $O(\epsilon T)$.

Problem 2.3: Diffusion Policy (30 pts)

2.3.1 Loss Plot + Final Loss value Diffusion Policy (6 pts)

Solution



Final loss: 0.0469

2.3.2 Rewards Diffusion Policy (15 pts)

2.3.2.1; 3 actions evaluated in a row (5 pts)

avg trajectory time: 9.28 s mean: 183.5 median: 261.1 max: 266.1

2.3.2.2; 2 actions evaluated in a row (5 pts)

avg trajectory time: 14.51 s mean: 202.6 median: 261.8 max: 266.8

2.3.2.3; 1 action evaluated in a row (5 pts)

avg trajectory time: 31.22 s mean: 231.3 median: 264.3 max: 268.9

2.3.3 GIF link Diffusion Policy (2 pts)

Solution

https://drive.google.com/file/d/10M9vmolRiQeCRepCSjUIh34KVuP8NDtT/view?usp=sharing

2.3.4 Compare diffusion policy and simple model runtime (written, 4 pts)

Solution

BC/DAgger's SimpleNet gets an action from a single forward pass of a small MLP. The diffusion policy instead has to denoise its actions: every time it chooses actions it runs 30 sequential denoising steps, and each step is a full forward pass of a 6-layer transformer decoder.
These steps cannot be parallelized since each depends on the previous one (sequentially) On our CPU one SimpleNet call took ≈0.04 ms versus ≈36.5 ms for one 30-step diffusion sample (∼1000× slower), which is why a diffusion trajectory takes 9–31 s.

2.3.5 Compare diffusion policy with different actions in a row runtime (written, 3 pts)

Solution

Runtime is dominated by diffusion samples, which cost the same regardless of N (≈36.5 ms each). Executing N predicted actions per sample means an episode of T steps needs only $\lceil T/N \rceil$ samples, so time per environment step scales as $1/N$, which results in 12.2 ms (N=3), 18.3 ms (N=2) and 38.0 ms (N=1), The average trajectory times (9.28, 14.51, 31.22 s) also depend on episode length (761, 791, 821 steps on average).